FIAP ON

DATA VIZ & DATA SCIENCE

MODELAGEM PREDITIVA

10

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Tipos de Analytics 5
Figura 2 – Diagrama de aprendizagem de máquina supervisionada 7
Figura 3 – CRISP-DM 9
Figura 4 – Exemplo árvore de probabilidades.....11
Figura 5 – Pipeline de ML 12
Figura 6 – Pipeline de ML – Etapas 13
Figura 7 – Dataset de vinho 14
Figura 8 – Comparação de valores de ratings previstos versus valores reais 19
Figura 9 – Inclusão das colunas de erro absoluto e erro relativo (percentual) 19
Figura 10 – Histograma para analisar a distribuição do erro.....20
Figura 11 – Gráfico de dispersão entre valores previstos e reais iguais 23
Figura 12 – Gráfico de dispersão entre valores previstos e reais.....23
Figura 13 – Gráfico de dispersão entre valores previstos e reais.....24
Figura 14 – Coeficiente de correlação linear entre valores previstos e reais 25
Figura 15 – Cálculo do erro quadrado.....26
Figura 16 – Cálculo de métricas de erro de previsão.....27
Figura 17 – Valores previstos e reais para um classificador 29
Figura 18 – Matriz de confusão com sklearn e crosstab 30
Figura 19 – Métricas Accuracy, Precision e Recall 32

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE QUADROS

Quadro 1 – Data-set modelo preditivo.....8

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Dataset de dados de avaliações de vinhos.....14
Código-fonte 2 – Pipeline de um algoritmo preditivo em *Python*.....17
Código-fonte 3 – Dataset de dados de avaliações de vinhos.....18
Código-fonte 4 – Cálculo do intervalo de confiança 22
Código-fonte 5 – Pipeline de um algoritmo preditivo em *Python*.....28

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

MODELAGEM PREDITIVA 5
1 INTRODUÇÃO 5
2 MODELAGEM PREDITIVA 7
2.1 Quanto ao tipo da aprendizagem 7
2.2 Quanto ao tipo da variável-alvo (Y) 8
2.3 Regressores e classificadores.....8
2.4 Pipeline de modelos preditivos.....9
2.5 Avaliação de modelos 11
3 MONTAGEM DE UM PIPELINE DE ML BÁSICO EM PYTHON 13
4 AVALIAÇÃO DE MODELOS 18
4.1 Regressores.....18
4.1.1 Intervalo de confiança 21
4.1.2 Interpretação gráfica do erro de previsão.....22
4.1.3 Métricas para cálculo do erro de previsão (Loss Function) 25
4.2 Classificadores.....27
4.2.1 Matriz de confusão 29
4.2.2 Métricas de avaliação em classificadores 31
REFERÊNCIAS.....33

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

MODELAGEM PREDITIVA

1 INTRODUÇÃO

A ideia é relativamente simples: queremos utilizar uma base de dados conhecida de um determinado evento para prever o que acontecerá num momento futuro, assumindo que as principais características nas quais o evento esteja envolvido sejam mantidas.

Quando falamos de tipos de análise de dados ou analytics, podemos classificar numa escala evolutiva em relação à decisão e ação, da seguinte forma:

- Análise descritiva: descreve o histórico do que aconteceu.
- Análise diagnóstica: traz os porquês de algo ter acontecido.
- Análise preditiva: procura trazer o que acontecerá, uma previsão.
- Análise prescritiva: traz ações que deveriam ser feitas.
- Inteligência artificial: automatização de ações humanas.

Podemos avaliar essa escala no diagrama da Figura “Tipos de Analytics”.

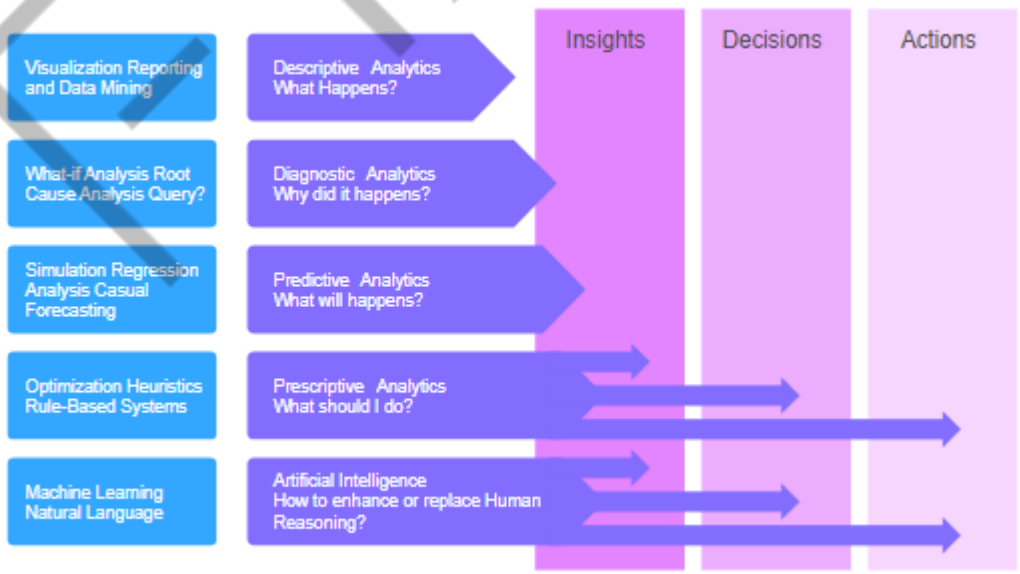

Figura 1 – Tipos de Analytics
Fonte: http://www.infme.com/descriptive-and-predictive-data-analytics-in-big-data/

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 MODELAGEM PREDITIVA

2.1 Quanto ao tipo da aprendizagem

Um modelo preditivo ou análise preditiva dizemos que é um tipo de aprendizagem supervisionada. Podemos observar o diagrama de um modelo supervisionado na Figura “Diagrama de aprendizagem de máquina supervisionada”.

Figura 2 – Diagrama de aprendizagem de máquina supervisionada
Fonte: Elaborado pelo autor

Para um modelo preditivo, necessitamos, então, de uma variável-alvo (Y) e uma ou mais variáveis preditoras (x). Também chamamos essas variáveis de:

- Variável-alvo: variável dependente, variável endógena ou simplesmente Y.
- Variáveis preditoras: variáveis independentes, variáveis exógenas, features (características) ou simplesmente X’s.

Precisamos, então, de um data-set no formato apresentado no quadro a seguir: “Data-set modelo preditivo”.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	Y	X1	...	Xn
Linha (ou Obs) 1	Y(1)	X1(1)	...	Xn(1)
Linha (ou Obs) 2	Y(2)	X1(2)	...	Xn(2)
⋮	⋮	⋮	⋮	⋮
Linha (ou Obs) n	Y(n)	X1(n)	...	Xn(n)

Quadro 1 – Data-set modelo preditivo
Fonte: Elaborado pelo autor.

2.2 Quanto ao tipo da variável-alvo (Y)

Em um modelo preditivo, começaremos analisando o tipo de evento que queremos prever, e já podemos dividi-lo em dois grandes grupos com base no tipo da variável que chamaremos de "alvo":

- Variável-alvo numérica: pode ser o caso em previsões de vendas, em que a variável-alvo será o montante total de vendas, um valor numérico.
- Variável-alvo categórica: temos inúmeras aplicações, sendo uma delas, por exemplo, previsão de churn.

2.3 Regressores e classificadores

Chamamos, respectivamente, de regressores, quando realizamos previsões de variáveis numéricas; e de classificadores, quando realizamos previsões de variáveis categóricas.

Alguns exemplos de regressores:

- Previsões de valores numéricos sobre quanto vamos atingir determinado nível ou gatilho limite.
- Previsões de vendas ou forecasting.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Como classificadores, podemos trazer:

- Previsão de churn: a variável-alvo é se o cliente abandonará ou não a empresa.
- Previsão de fraude: a variável-alvo é uma transação boa ou uma fraude.
- Classificador de imagens: em um carro autônomo, responde se o veículo deve acelerar ou parar.

2.4 Pipeline de modelos preditivos

O diagrama CRISP-DM pode nos ajudar a entender um pipeline ou uma esteira de modelos preditivos. Vejamos a Figura “CRISP-DM”.

Figura 3 – CRISP-DM
Fonte: https://web.archive.org/web/20060624054212/http://www.crisp-dm.org/News/86605.pdf

Vamos entender, agora, cada uma das etapas:

1. Entendimento de negócio: é de suma importância que iniciemos entendendo nosso objetivo, o que nos guiará durante o processo. Precisamos responder o que estamos querendo prever, no caso de uma análise preditiva. Também

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

é nesta etapa que entendemos se vamos trabalhar com regressor ou classificador, após definir qual será nossa variável-alvo.

2. Entendimento dos dados: nesta etapa precisamos entender quais dados serão necessários para atingir o objetivo, ou seja, construir a análise preditiva. Já possuímos estes dados ou precisaremos obtê-los? É necessário obter dados adicionais?
3. Preparação dos dados: aqui entraram desde limpezas de dados (dados vazios, inconsistentes ou duplicados, por exemplo) até transformações de variáveis que se fizerem necessárias. Podemos transformar uma variável idade em faixas etárias, por exemplo. Em *machine learning*, também costumamos chamar esta etapa de feature engineer, dado que estamos trabalhando na engenharia (construção, transformação etc.) das features (ou variáveis preditoras).
4. Modelagem estatística: chamamos de modelagem simplesmente, e é aqui que construímos modelos estatísticos. Chamamos de modelos estatísticos, pois eles não são modelos com fórmulas exatas, precisas; como são estimativas de futuros, sempre há um erro (estatístico) associado, inerente ao processo.
5. Avaliação do modelo: após a construção do modelo, precisamos avaliar se este modelo tem um erro estatístico estimado aceitável para o problema que estamos querendo resolver. É de suma importância, nesta etapa, que saibamos como analisar os erros. Você pode notar que há uma seta no CRISP-DM que leva para a etapa inicial, tornando este conjunto de tarefas cíclico. E isto é comum acontecer, pois testamos (avaliamos) vários modelos, às vezes, dezenas, centenas e até milhares.
6. Deploy: esta etapa significa que decidimos utilizar um modelo "em produção", ou seja, para realizar uma atividade de previsão. Isto implica decidir como este modelo, que é uma "fórmula de cálculo" implementada em um formato de função computacional (dados que entram e uma saída, que é a previsão), será colocado em execução. Isto envolve questões de engenharia de software[1].

[1] Atualmente, temos esteiras de deploy que são camadas de MLOps, similarmente à terminologia utilizada DevOps, que significa resumidamente colocar Development (desenvolvimento de software) em operação (Ops). O acrônimo MlOps significa MachineLearning to Operation.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Veja este outro diagrama da Figura “EMachine learning pipeline” e procure analisar a correspondência com o CRISP-DM (são muito similares em essência).

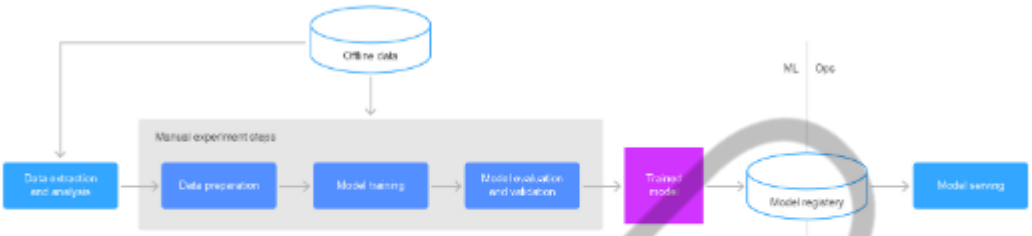

Figura 4 – Machine Learning Pipeline
Fonte: https://valohai.com/machine-learning-pipeline/

2.5 Avaliação de modelos

Vamos detalhar um pouco mais essa etapa para entender como pode ser avaliado um modelo preditivo. De uma forma bem resumida, podemos dizer que um modelo preditivo realiza previsões de eventos futuros. E nosso objetivo é realizar boas previsões, tão melhores quanto possível. Mas o que é uma "boa previsão"? Ora, é quando "acertamos" uma previsão ou, de outra forma, quando não erramos ou nosso erro é bem pequeno.

Então é fundamental que analisemos o erro de previsão em modelos preditivos. Podemos avaliar um terceiro diagrama, que detalha um pouco mais algumas das etapas do CRISP-DM e permite que avaliemos como aplicar a análise de erro.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

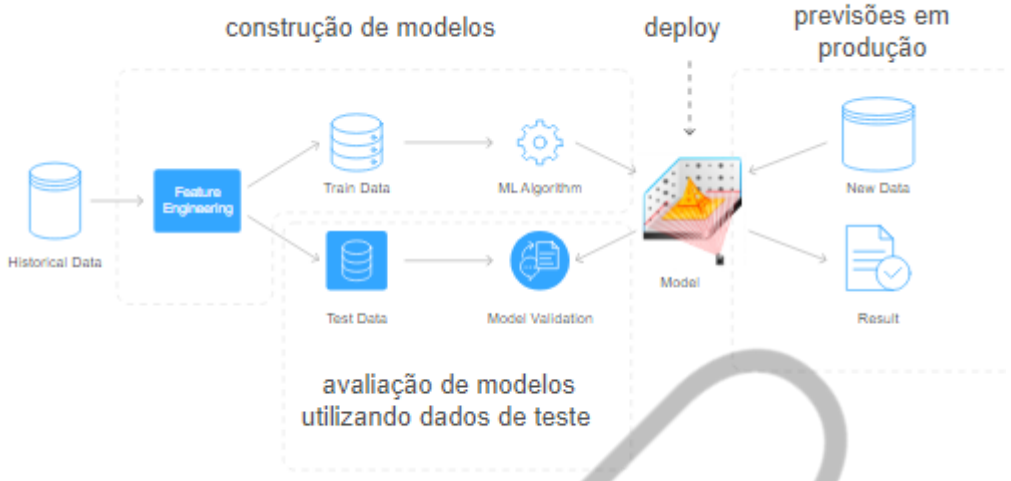

Figura 5 – Pipeline de ML
Fonte: https://commandstech.com/ml/

Na Figura “Pipeline de ML”, temos um elemento essencial: um data-set chamado "Test Data". Este data-set nada mais é do que parte dos dados históricos que foram separados previamente para a construção do modelo. Repare que o modelo é construído com outro dataset chamado Train Data (dados de treinamento).

Uma característica-chave sobre Train e Test Data: eles são conjuntos de dados mutuamente excludentes. Ou seja, uma observação (ou linha) que está em um data-set, não está em outro.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3 MONTAGEM DE UM PIPELINE DE ML BÁSICO EM PYTHON

Vamos nos concentrar, agora, nas etapas de montagem de um pipeline básico de *machine learning* para modelos preditivos utilizando *Python*. Vamos separar nas seguintes etapas:

1. Carga das bibliotecas a serem utilizadas.
2. Carga dos dados.
3. Preparação dos dados.
4. Separação das amostras de treino e teste.
5. Construção do modelo.
6. Avaliação do modelo.

Para melhor orientação visual, vamos avaliar novamente a figura anterior, porém, apontando cada uma das etapas, o que pode ser visto na Figura “Pipeline de ML – Etapas”.

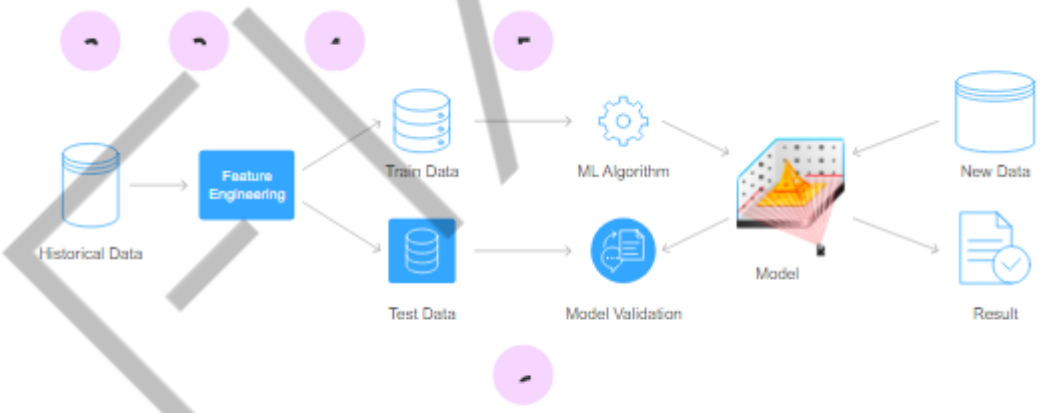

Figura 6 – Pipeline de ML – Etapas
Fonte: https://commandstech.com/ml/

Traduzindo essas etapas para *Python*, vamos utilizar o Colab para construir esse pipeline básico. Primeiramente, vamos conhecer o nosso conjunto de dados e qual a análise preditiva que estamos montando.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Escolhemos, para este exemplo, uma base de dados pública de vinhos tintos[2] e suas avaliações (*ratings*) que estão armazenadas no site Kaggle. Ela foi construída a partir de uma etapa de *webscrapping* do site Vivino. A plataforma Vivino é utilizada por apreciadores de vinho do mundo inteiro. Após provar um vinho, o usuário pode avaliá-lo. Esta avaliação pode ser útil para outras pessoas no momento da compra de um novo vinho, não somente para saber sua nota, ou seja, se o vinho é bom ou ruim, mas também para conhecer as características, saber como combinar com refeições, entre outras opiniões.

```
import pandas as pd
df =
pd.read_csv("https://raw.githubusercontent.com/diogenesjusto/FIAP/mast
er/dados/wine/red_wine4_30.csv", encoding='latin-1')
pd.head()
```

Código-fonte 1 – Dataset de dados de avaliações de vinhos
Fonte: Elaborado pelo autor (2022)

	Unnamed: 0	wine_name	wine_year	wine_country	wine_region	winery	wine_rating	n_ratings	wine_price
0	1	Merlot	2019	United States	California	Crane Lake	3.6	30	4.99
1	2	Cabernet Sauvignon	2011	United States	Columbia Valley	Hogue	3.3	76	4.95
2	3	Hot To Trot Smooth Red Blend	2013	United States	Columbia Valley	14 Hands	3.7	3917	4.99
3	4	Cabernet Sauvignon	2010	Chile	Maule Valley	Aromo	3.0	189	4.99
4	5	Loggia del Conte Chianti	2016	Italy	Chianti	Le Chiantigiane	3.3	149	4.99
5	6	California Zinfandel	2009	United States	California	A J	3.5	25	4.99
6	7	Tinto	2017	Portugal	Tejo	Xairel	3.2	49	4.25
7	8	Merlot	2015	United States	California	Crane Lake	3.1	198	4.99
8	9	Pinot Noir	2016	United States	California	Coastal Vines Cellars	3.5	128	4.99
9	10	Cabernet Sauvignon	2018	Chile	Central Valley	Frontera	3.3	1596	5.00
10	11	Syrah	2007	United States	Santa Barbara County	Kinton Vineyards	3.9	73	4.95
11	12	Cabernet Sauvignon	2019	Chile	Central Valley	Fuego Austral	3.0	50	4.99
12	13	Cabernet Sauvignon - Merlot	2020	Chile	Central Valley	Frontera	3.5	74	4.99
13	14	Malbec	2011	Argentina	Mendoza	Funky Llama	3.1	57	4.99
14	15	Cabernet Sauvignon	2016	United States	California	Crane Lake	3.4	325	4.99

Figura 7 – Dataset de vinho
Fonte: Elaborado pelo autor (2022)

1. Em nosso exemplo, vamos elaborar uma análise preditiva para prever a nota de um vinho baseado na safra e no preço. Isso pode ser útil, por exemplo, se não tivermos aquele rótulo previamente classificado por outros usuários3. Seguimos com a construção do pipeline desta forma: Carga das bibliotecas a serem utilizadas:

[2] https://www.kaggle.com/datasets/12fdf497115dabef31fb36f2f7c954e1217107710ec80c4a915657dc05a9dd03
[3] Esta funcionalidade ainda não existe na plataforma Vivino.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

   a. pandas: para trabalhar com dataframes em *Python.*
   b. sklearn:
      i. model_selection: utilizaremos, na etapa 4, separação de dados em treino e teste[4].
      ii. linear_model: é o algoritmo/modelo escolhido aqui para exemplificação (veremos na próxima seção que é um regressor).
      iii. metrics: é a biblioteca de métricas de avaliação de modelos, sendo que utilizaremos, neste exemplo, uma específica.
2. Carga dos dados:
   a. Faremos a carga de um csv diretamente do github em formato raw (dados não tratados ou dados "crus").
   b. Você notará um parâmetro do tipo "encoding". Neste caso, este parâmetro foi necessário, pois o dataset foi salvo com um encoding específico, diferente do padrão utilizado no Colab.
3. Preparação dos dados:
   a. Neste primeiro exemplo, utilizaremos os dados como estão, sem nenhum tratamento. Deixaremos esta seção presente somente como um comentário por questões de organização do código.
4. Separação das amostras de treino e teste:
   a. Utilizaremos a função train_test_split que permite, em uma única linha, a definição de 4 objetos de saída, que é uma premissa de utilização do sklearn em modelos. Teremos, na saída 2, objetos para dados de treino (x e y, que são as features ou variáveis preditoras e *target* ou variável-alvo, respectivamente). Teremos também 2 objetos para dados de teste que seguem a mesma ideia dos dados de treino[5].

[4] Em inglês esta etapa também é chamada de *sampling*, que significa amostragem ou *split*, que significa quebra ou separação em partes.
[5] Não é muito relembrar que os dataset de treino e teste são mutuamente excludentes, ou seja, dados que estão em um não estão em outro. Você pode inspecionar os dataframes para confirmar isso. Uma forma de aferir ao menos as quantidades de dados é utilizar o comando x_treino.shape, que deve retornar o número de linhas e colunas deste dataframe, e fazer o mesmo com os demais. Por lógica, o número de linhas de x_treino deve ser igual a y_treino, valendo também para teste respectivamente. E se somarmos o número de linhas de x_treino com o número de linhas de x_teste, o resultado deverá ser igual ao número de linhas do dataset original (df.shape para conferir). Claro que esta simples aferição não garante que temos dados mutuamente excludentes. Seria necessária uma aferição adicional.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

b. Repare que, neste momento, escolhemos qual é a nossa variável-alvo[6], que, no caso, será "wine_rating", e é uma variável numérica (os ratings são valores numéricos ordenados entre 0 e 6, com intervalos de 0,1, ou seja, 1 décimo).
c. A função train_test_split pede mais dois argumentos:
   i. test_size: 0.3266 significa que do total de linhas no dataset original df serão utilizados 32,66% para a base teste (e o restante para base treino). Em geral, utilizamos uma proporção uma pouco maior para dados de treino do que dados de teste[7].
   ii. random_state: a função train_test_split realizando uma *sampling* (amostragem) aleatória dos dados. Ou seja, embaralha a ordem original das linhas antes de separar treino e teste. O parâmetro 33 aqui é utilizado para que consigamos reproduzir esta amostragem quando for necessário, se quisermos uma amostragem com distribuição aleatória de linhas diferente, para escolher outro número qualquer que não seja o 33.

5. Construção do modelo:
   a. Neste caso específico, utilizaremos um modelo de regressão linear, que é um regressor, ou seja, aplicável quando temos uma variável-alvo do tipo numérico, que é nosso caso.
   b. O comando .fit é o que ajusta o modelo escolhido (no caso, uma reta dada por uma regressão linear) a um conjunto de pontos conhecidos (x e y).
6. Avaliação do modelo (sobre a base teste):
   a. Nesta etapa, precisamos utilizar o modelo criado anteriormente, que nomeamos de mod, passando para esta "função" os dados de entrada que são as features (x) do conjunto de teste. Como resultado, teremos a variável y_prev, que são as previsões de *ratings* de vinhos baseados no ano (safra) e no preço.

[6] Relembre que esta variável y também é chamada de *target,* variável dependente, variável endógena, que são outros nomes também utilizáveis em outros campos de estudo. É interessante reforçar estas nomenclaturas, pois, em bibliografia, livros e sites especializados, é comum utilizar-se destes nomes diferentes.
[7] Ainda neste curso, serão exploradas as técnicas de *sampling* e *cross-validation*.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

b. A função mean_squared_error foi escolhida para calcular o erro dessa previsão ao comparar as variáveis y_teste (ou seja, os dados reais de *ratings* presentes no dataset de teste) e y_prev (ou seja, os dados previstos para *rating*).

```
# 1 - Carga das bibliotecas
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 2 - Carga dos dados
df =
pd.read_csv("https://raw.githubusercontent.com/diogenesjusto/FIAP/mast
er/dados/wine/red_wine4_30.csv", encoding='latin-1')

# 3 - Preparação dos dados

# 4 - Separação das amostras de treino e teste
x_treino, x_teste, y_treino, y_teste =
train_test_split(df[['wine_year', 'wine_price']],
df['wine_rating'],test_size=0.3266, random_state=33)

# 5 - Construção do modelo
mod = LinearRegression()
mod.fit(x_treino,y_treino)

# 6 - Avaliação do modelo
y_prev = mod.predict(x_teste)
mean_squared_error(y_teste, y_prev)
```

Código-fonte 2 – Pipeline de um algoritmo preditivo em *Python*
Fonte: Elaborado pelo autor (2022)

A avaliação de modelos preditivos tem características diferentes quando analisamos erros de previsão para variáveis-alvo numéricas e variáveis-alvo categóricas. Vamos ver cada um desses tipos, respectivamente, nas próximas 2 seções.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

4 AVALIAÇÃO DE MODELOS

Basicamente, em qualquer modelo preditivo realizamos previsões. E a previsão é "boa", quando acertamos, e “ruim”, quando erramos. Mas a resposta não é tão simples quando acertar ou errar. Para isso, utilizamos métricas para calcular erros de previsão e analisaremos se esses erros são de magnitude grande ou pequena.

Em aprendizado de máquina, utilizamos o termo função perda (*loss function*) para definir uma função (ou fórmula) de cálculo para "quanto perdemos", para quando tentamos fazer uma previsão. A perda é justamente o erro de previsão. Dizemos então que queremos desenvolver algoritmos que minimizem a função perda (ou tenha um erro reduzido).

Vamos analisar, agora, como interpretamos erros de previsão de modelos preditivos tanto regressores quanto classificadores, além de conhecer algumas das possíveis funções de perda ou métricas de erro de previsão.

4.1 Regressores

A denominação geral Regressor é utilizada quando estamos aplicando técnicas de análise preditiva ou modelos de aprendizagem supervisionada, em que a variável y é do tipo numérica contínua[8].

Vamos começar analisando parte dos dados de teste reais versus as previsões que construímos.

```
ev = pd.DataFrame({"prev": y_prev, "real": y_teste})
ev.head(n=10)
```

Código-fonte 3 – Dataset de dados de avaliações de vinhos
Fonte: Elaborado pelo autor (2022)

[8] No exemplo anterior, você deve ter reparado que a variável *rating* não é exatamente uma variável contínua, pois ela não permite valores intermediários entre duas frações decimais (por exemplo 3.5 e 3.6). Mas, neste caso, estamos utilizando uma aproximação que não deve penalizar em muito.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	prev	real
253	3.534321	3.5
2052	3.648876	3.9
3695	3.715762	3.3
630	3.592574	3.9
6594	3.847883	3.7
5958	3.787866	3.7
6489	3.833733	3.8
9552	3.913194	3.9
6112	3.795111	3.6
8700	3.920812	3.9

Figura 8 – Comparação de valores de ratings previstos versus valores reais
Fonte: Elaborado pelo autor (2022)

Quando analisamos a Figura “Comparação de valores de ratings previstos versus valores reais”, reparamos que os valores previstos são próximos aos valores reais, sendo que alguns são mais próximos e outros mais distantes. Essa distância, que nada mais é do que a diferença, é o erro de previsão. Vamos incluir esse cálculo para avaliar o erro em cada linha e vamos incluir o cálculo de proporção deste erro, ou seja, o valor percentual do erro em relação ao valor real.

```
ev['erro_absoluto'] = y_prev-y_teste
ev['erro_relativo'] = (y_prev-y_teste)/y_teste
ev.head(n=10)
```

	prev	real	erro_absoluto	erro_relativo
253	3.534321	3.5	0.034321	0.009806
2052	3.648876	3.9	-0.251124	-0.064391
3695	3.715762	3.3	0.415762	0.125988
630	3.592574	3.9	-0.307426	-0.078827
6594	3.847883	3.7	0.147883	0.039968
5958	3.787866	3.7	0.087866	0.023748
6489	3.833733	3.8	0.033733	0.008877
9552	3.913194	3.9	0.013194	0.003383
6112	3.795111	3.6	0.195111	0.054197
8700	3.920812	3.9	0.020812	0.005337

Figura 9 – Inclusão das colunas de erro absoluto e erro relativo (percentual)
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Interessante notar que alguns rótulos conseguimos prever com valores muito próximos ao real, caso das linhas 253 (que está no topo), 6489, 9552 e 8700[9]. Para essas quatro linhas (ou estes quatro rótulos de vinho), nosso erro foi menor que 1%, sendo 0,33% o menor erro. Mas alguns outros rótulos tiveram erros altos, como é o caso da linha 3695, com um erro de 12,59%.

Outro fator interessante a observar é que temos erros negativos. Isso ocorre, pois nosso algoritmo preditivo, às vezes, "erra para cima" e, outras vezes, "erra para baixo".

Avaliar linha a linha cada erro é interessante para uma análise qualitativa, mas temos milhares de linhas para analisar. Precisamos de outras ferramentas mais eficientes para analisar erros de previsão e o histograma é uma ferramenta excelente para analisar a distribuição de erros. Queremos responder a esta pergunta: estamos errando mais para cima ou mais para baixo?

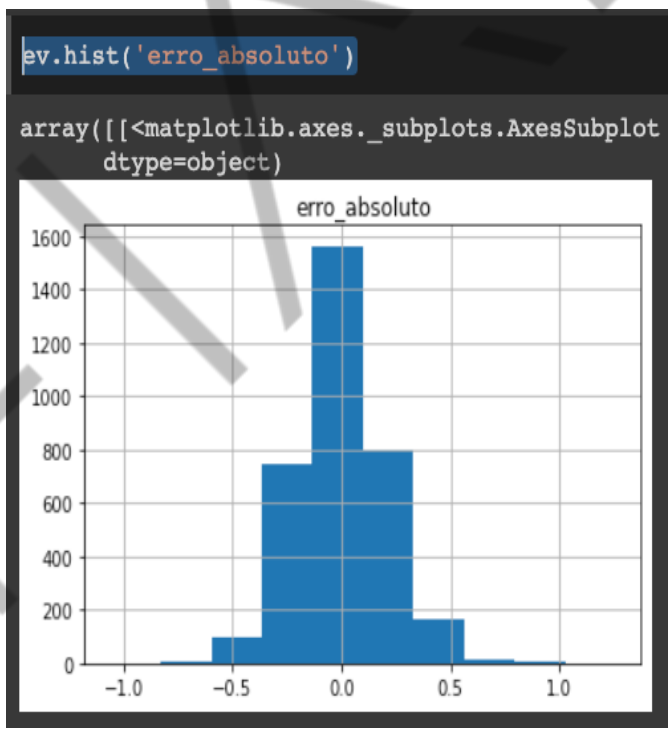

Figura 10 – Histograma para analisar a distribuição do erro
Fonte: Elaborado pelo autor (2022)

[9] Chamou a atenção que os números de linhas não estão ordenados? Eles parecem embaralhados, certo? Pois isto é consequência justamente da aplicação da função train_test_split e é uma ação necessária para termos amostras distribuídas aleatoriamente, ou seja, não termos dados enviesados em uma amostra de treino ou teste

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Notamos que o histograma tem uma distribuição que lembra a conhecida curva normal ou gaussiana, e repare que o centro da distribuição é 0. Podemos interpretar esse histograma dizendo que, na grande maioria das vezes (das predições), nosso algoritmo preditivo tem um erro entre -0,5 e 0,5. Reparem que escolhemos arbitrariamente o intervalo -0,5 e 0,5, talvez justamente por esses valores já estarem demonstrados no eixo do gráfico.

Uma próxima pergunta seria: mas isto é bom ou ruim?

Ou ainda: estas previsões, com estes erros demonstrados, são úteis para a aplicação que estou propondo? Na maioria das vezes, as minhas previsões têm um erro aceitável baixo? E quanto é esta "maioria"?

Para responder a essas perguntas, resgataremos mais conceitos de estatística básica (além do histograma que já utilizamos), que são adequados para responder perguntas sobre incerteza.

4.1.1 Intervalo de confiança

Para responder às perguntas acima, vamos começar relembrando o que é o intervalo de confiança.

Na seção intervalo, definimos um intervalo de confiança entre -0,5 e 0,5. Lembre-se de que este 0,5 é o erro relativo a uma previsão da variável *rating* de um vinho que pode receber *ratings* entre 0 e 5.

Visualmente, pelo histograma, podemos perceber que ficam degraus pequenos no lado esquerdo e lado direito do histograma, fora deste intervalo. Mas não sabemos com exatidão quantas previsões estão dentro desse intervalo.

Para calcular o intervalo de confiança, vamos escolher primeiro o que é o intervalo que queremos, isto é, qual é o % de confiança que queremos obter. Pratica-se, muitas vezes, uma escolha de 95% de confiança, que seria, em termos de observações, em 100 previsões termos 95 dentro do intervalo que calculamos. As outras 5 podem ter um erro maior.

Vamos voltar ao nosso exemplo e fazer uso de um bibliotece scypy.stats para nos auxiliar neste cálculo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
import numpy as np
import scipy.stats as st

#create 95% confidence interval for population mean weight
data = ev[['erro_absoluto']].to_numpy()
ci = st.t.interval(alpha=0.95, df=len(ev.index)-1,
loc=np.mean((data)), scale=np.std(data))
print(ci)
```

Código-fonte 4 – Cálculo do intervalo de confiança
Fonte: Elaborado pelo autor (2022)

Essa função estatística é relativamente simples de calcular, porém como já temos uma biblioteca com ela implementada, faz sentido utilizá-la.

O resultado é: (-0.3974767426788493, 0.39480351539171527)

Isso significa dizer que 95% das previsões que fizermos terão um erro menor que 0,39747 para cima ou para baixo. O parâmetro alpha, no Código-fonte “Cálculo do intervalo de confiança”, serve para ajustarmos o % do intervalo de confiança que queremos calcular.

4.1.2 Interpretação gráfica do erro de previsão

Além do uso do histograma e intervalo de confiança, é interessante analisarmos graficamente valores previstos versus valores reais.

Numa situação idealizada em que tivéssemos valores previstos muito próximos dos valores reais e realizássemos um gráfico de dispersão entre essas duas variáveis, o que deveríamos esperar? Vejamos a Figura” Gráfico de dispersão entre valores previstos e reais iguais”.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Gráfico de dispersão consideração situação "ideal",
# > valores previstos iguais ao reais
ev.plot.scatter(x="prev", y="prev")
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f5067dc9750>
```

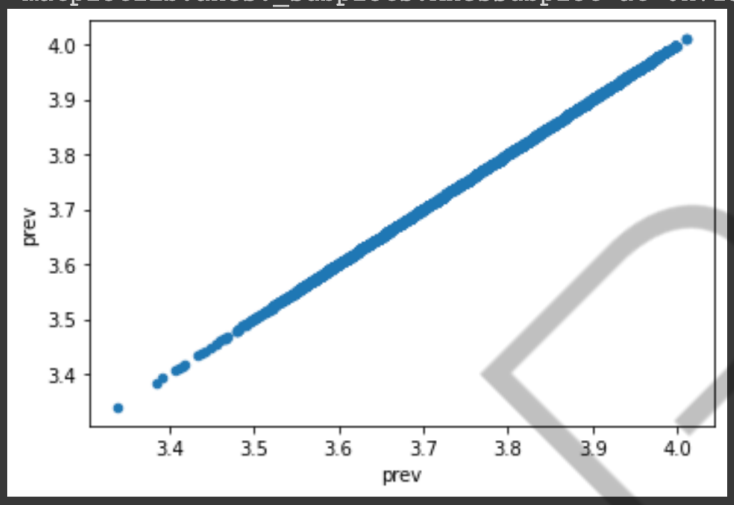

Figura 11 – Gráfico de dispersão entre valores previstos e reais iguais
Fonte: Elaborado pelo autor (2022)

Ou seja, quanto mais os valores previstos x reais se parecerem com uma reta com inclinação positiva, mais próximos eles serão. Vamos então analisar, através de um gráfico similar, os valores previstos e reais do nosso exemplo anterior, na Figura "Gráfico de dispersão entre valores previstos e reais iguais”.

```
# Gráfico de dispersão previsto x real
ev.plot.scatter(x="prev", y="real")
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7ff2c40cca50>
```

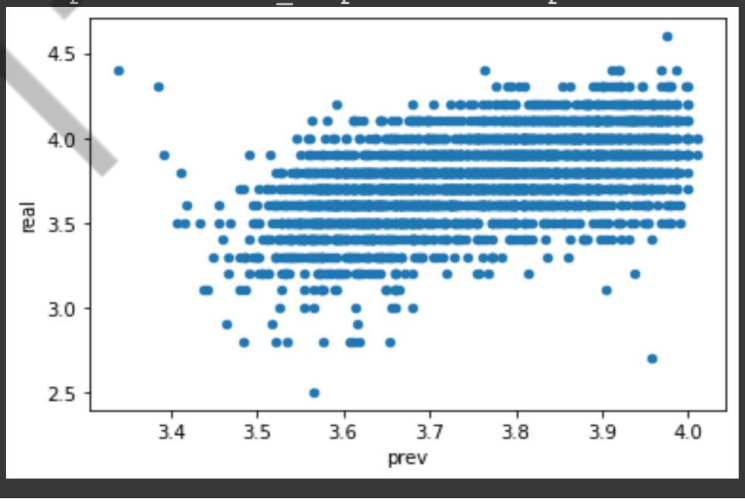

Figura 12 – Gráfico de dispersão entre valores previstos e reais
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Não é tão simples construir um algoritmo que tenha previsões muito boas e que esse gráfico seja muito próximo ao anterior. Mas a parte positiva é que esta "bolha" ou "borrão" ainda lembra alguma coisa como uma reta inclinada. O problema desse gráfico, no momento, é que temos muitas observações, mais de 3.000 pontos na tela[10]. Como temos muitos pontos se sobrepondo, é difícil visualizar onde está a maior concentração de pontos (densidade). Vamos utilizar um outro recurso gráfico que nos permite ver isso na tela.

Utilizaremos uma biblioteca chamada mpl_scatter_density, que permite construir um gráfico de dispersão a(*scatter*) apresentando a densidade dos pontos. Como essa biblioteca não está instalada no Colab, precisaremos adicioná-la com o comando !pip install antes de utilizá-la.

```
[13] !pip install mpl_scatter_density
     import mpl_scatter_density

     Installing collected packages: fast-histogram, mpl-scatter-density
     Successfully installed fast-histogram-0.11 mpl-scatter-density-0.7

[14] import matplotlib.pyplot as plt

     fig = plt.figure()
     ax = fig.add_subplot(1, 1, 1, projection='scatter_density')
     ax.scatter_density(y_prev, y_teste, color='blue')

     /usr/local/lib/python3.7/dist-packages/mpl_scatter_density/generic_d
       vmin = self._density_vmin(array)
     /usr/local/lib/python3.7/dist-packages/mpl_scatter_density/generic_d
       vmax = self._density_vmax(array)
     <mpl_scatter_density.scatter_density_artist.ScatterDensityArtist at
```

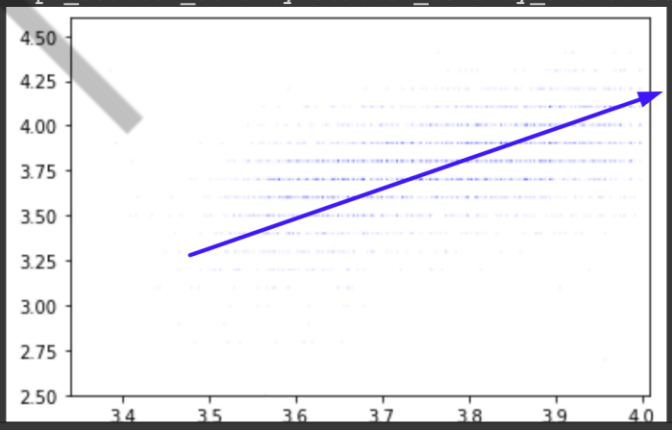

Figura 13 – Gráfico de dispersão entre valores previstos e reais
Fonte: Elaborado pelo autor (2022)

[10] Verifique isto com o comando ev.shape()

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Na Figura “Gráfico de dispersão entre valores previstos e reais”, observamos, que há uma cor mais escura no "miolo" da nuvem de pontos; procuramos sinalizar, por meio de uma seta azul, a orientação e a posição onde estão os pontos mais escuros e podemos concluir que há uma correlação entre estes valores. É claro que poderíamos também calcular essa correlação (linear) com o cálculo do coeficiente de correlação linear. Na Figura “Coeficiente de correlação linear entre valores previstos e reais”, observamos que 0,53 não é lá uma correlação linear alta.

```
# Coeficiente de correlação linear
ev[['prev','real']].corr()
```

	prev	real
prev	1.000000	0.530011
real	0.530011	1.000000

Figura 14 – Coeficiente de correlação linear entre valores previstos e reais
Fonte: Elaborado pelo autor (2022)

4.1.3 Métricas para cálculo do erro de previsão (Loss Function)

Existem muitas métricas para cálculo de erro de previsão de regressores; a escolha de qual utilizar dependerá muito do problema que estamos resolvendo, o que estamos buscando. É interessante conhecer algumas para que tenhamos noção da diversidade das métricas.

Inicialmente, retornamos à tabela da Figura “Inclusão das colunas de erro absoluto e erro relativo (percentual)”, onde temos 2 colunas chamadas erro_absoluto e erro_relativo. Nessa coluna, temos um valor de erro absoluto e percentual calculado para cada linha (cada previsão). Mas se vamos analisar a performance do algoritmo como um todo, ao realizar previsões, precisaríamos gerar uma métrica baseada em todos esses valores de erro.

Que tal calcular a média? Ou seja, calcular o erro médio?

É uma boa abordagem, porém reparem que temos erros positivos e erros negativos. O que acontece ao calcular a média com valores negativos e positivos presentes na mesma variável? Eles se anulam, pois a fórmula de cálculo da média é

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

a soma de todos valores e posterior divisão pela quantidade de valores. Precisamos, de alguma forma, então, "eliminar" o efeito do valor negativo. Uma forma de fazer isso é elevar ao quadrado, pois números negativos quando multiplicados por eles mesmos, tornam-se positivos.

```
# Cálculo do erro quadrático
ev['erro_quadrado']=ev['erro_absoluto']**2
ev
```

	prev	real	erro_absoluto	erro_relativo	erro_quadrado
253	3.534321	3.5	0.034321	0.009806	0.001178
2052	3.648876	3.9	-0.251124	-0.064391	0.063063
3695	3.715762	3.3	0.415762	0.125988	0.172858
630	3.592574	3.9	-0.307426	-0.078827	0.094511
6594	3.847883	3.7	0.147883	0.039968	0.021869
...	...	...	...	...	...
4265	3.754047	3.6	0.154047	0.042791	0.023730
1556	3.630109	3.6	0.030109	0.008364	0.000907
8806	3.907714	3.7	0.207714	0.056139	0.043145
4461	3.765382	3.4	0.365382	0.107465	0.133504
915	3.545921	3.5	0.045921	0.013120	0.002109

Figura 15 – Cálculo do erro quadrado
Fonte: Elaborado pelo autor (2022)

Pronto! Agora podemos fazer cálculos com a coluna erro_quadrado sem nos preocuparmos com valores que se anulam. Como citamos anteriormente, existem diversas métricas de cálculos de erros de previsão (função perda) para regressores. Nosso objetivo aqui não será cobrir todos possíveis, mas citar alguns.

Poderíamos somar essa coluna. Como resultado, quanto menor for o valor, menor será o erro de previsão. Essa métrica se chama soma do erro quadrático ou *Sum of Squared Error*, em inglês, que se abrevia por SSE.

Poderíamos calcular a média dessa coluna. Como resultado também quanto menor for o valor dessa métrica, menor será o erro. A essa métrica damos o nome de média do erro quadrático ou *Mean of Squared Error*, em inglês, ou simplesmente MSE.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Como calculamos o quadrado, na métrica anterior MSE, pode ser interessante trazer essa métrica para a ordem de grande da variável original aplicando a função inversa ao quadrado, que é a raiz quadrada. Teríamos, então, a raiz quadrada da média do erro quadrado ou *Root Mean Squared Error,* em inglês, ou simplesmente RMSE.

Temos muitas formas diferentes de calcular erros de regressores, cada qual com uma característica específica e aplicável em uma determinada situação. Vejamos, na Figura “Cálculo de métricas de erro de previsão”, os cálculos dessas 3 métricas de formas diferentes, utilizando fórmula de cálculo e através da biblioteca específica do sklearn; o resultado é o mesmo. Exemplificamos aqui duas formas para demonstrar que as funções do sklearn nada mais são do que fórmulas previamente organizadas na biblioteca para facilitar o trabalho do analista.

```
# Cálculo de Métricas de Erro
SSE = sum(ev['erro_quadrado'])
MSE = ev['erro_quadrado'].mean()
RMSE = np.sqrt(MSE)

# Cálculo do MSE utilizando o sklearn (repare que já utilizamos esta função anteriormente)
MSE_sklearn = mean_squared_error(y_teste, y_prev)

[SSE, MSE, MSE_sklearn,RMSE]
```

```
[138.47337022572387,
 0.04082351716560254,
 0.04082351716560254,
 0.20204830404040156]
```

Figura 16 – Cálculo de métricas de erro de previsão
Fonte: Elaborado pelo autor (2022)

A razão de termos incluído, nesse exemplo, o MSE calculado de duas formas é para demonstrar que a fórmula é simples e calculável. Repare que o sklearn (até a versão 1.1.1) ainda não tinha o RMSE e SSE implementados, porém ao consultar a página de referência do sklearn, você poderá verificar que já temos outras métricas de erro implementadas como função.

4.2 Classificadores

O que muda significativamente da avaliação de um regressor para um classificador é que não temos mais uma variável-alvo numérica; um classificador

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

classifica classes ou categorias. Vamos ver um exemplo para tornar mais fácil a compreensão?

```
# 1 - Carga das bibliotecas
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn import tree
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score

# 2 - Carga dos dados
df =
pd.read_csv("https://raw.githubusercontent.com/diogenesjusto/FIAP-On-
Line-Graduation/main/datasets/arvore_titanic.csv")

# 3 - Preparação dos dados

# 4 - Separação das amostras de treino e teste
x_treino, x_teste, y_treino, y_teste =
train_test_split(df[['Pclass']], df['Survived'],test_size=0.3266,
random_state=33)

# 5 - Construção do modelo
mod = tree.DecisionTreeClassifier()
mod.fit(x_treino,y_treino)

# 6 - Avaliação do modelo
y_prev = mod.predict(x_teste)
accuracy_score(y_teste, y_prev)
```

Código-fonte 5 – Pipeline de um algoritmo preditivo em *Python*
Fonte: Elaborado pelo autor (2022)

No Código-fonte “Pipeline de um algoritmo preditivo em Python”, podemos observar um código similar ao que estruturamos para o modelo preditivo de um regressor. A ideia e estrutura básica do código é a mesma; todas as mesmas seções de código estão ali presentes. Muda, no entanto, o fato de que a variável-alvo, y, agora é uma variável categórica e, por essa razão, a métrica de avaliação será diferente.

Neste exemplo colocamos o cálculo da acurácia. Vejamos uma tabela com os valores previstos e reais da variável Y.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
ev = pd.DataFrame({"prev": y_prev, "real": y_teste})
ev.head(n=10)
```

	prev	real
297	1	0
71	0	0
631	0	0
567	0	0
21	0	1
187	1	1
607	1	1
437	0	1
686	0	0
417	0	1

Figura 17 – Valores previstos e reais para um classificador
Fonte: Elaborado pelo autor (2022)

Repare que neste caso, analisaremos os erros e acertos de uma forma um pouco diferente. Faz sentido calcularmos a diferença entre Y previsto e Y real como sendo uma métrica de erro? O que pode ser considerado um erro neste caso?

Ora, um erro será computado para a linha em que tivermos o valor previsto diferente do valor real. E quando forem iguais, teremos um acerto. Simples, não? Não há, neste caso, acerto ou erro parcial para cada linha. A resposta só pode ser acerto ou erro.

Em uma aplicação de um sistema antifraude, quando recebemos, por exemplo, uma transação em nosso sistema de e-commerce, precisamos tomar uma decisão se barramos ou não a transação. Não há um meio-termo ou talvez.

4.2.1 Matriz de confusão

Uma forma interessante de analisarmos os dados previstos versus reais de classificadores é por meio de uma tabela cruzada, que denominamos de matriz de confusão (ou em inglês *confusion matrix*). Não há uma convenção se os dados previstos devem estar na orientação das colunas ou linhas. É importante, portanto,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

que avaliemos inicialmente, quando nos depararmos com uma matriz de confusão, qual é a definição utilizada.

Geramos a matriz de confusão de duas formas para este caso, para que você possa avaliar qual está mais adequada para sua análise; primeiramente utilizando a biblioteca sklearn e, posteriormente, simplesmente montando uma matriz "*crosstab*".

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_teste, y_prev)
```

```
array([[156,  27],
       [ 67,  42]])
```

```
# Matriz de Confusão
pd.crosstab(y_prev,y_teste, margins=True)
```

Survived	0	1	All
row_0			
0	156	67	223
1	27	42	69
All	183	109	292

Figura 18 – Matriz de confusão com sklearn e crosstab
Fonte: Elaborado pelo autor (2022)

Veremos algumas aplicações de matrizes de confusão e as interpretações delas.

Vamos nos fixar no caso de um classificador binário, que é bastante comum, para exemplificar as métricas associadas a uma matriz de confusão. Analisando a matriz referência, podemos observar que temos 4 células, assim denominadas e interpretadas:

- TP: True Positive - é a quantidade de observações previstas que bateram com os dados reais, para valores positivos da variável-alvo.
- TN: True Negative - é a quantidade de observações previstas que bateram com os dados reais, porém de valores negativos da variável-alvo.
- FP: False Positive - é a quantidade de erros quando tivemos uma previsão positiva, mas que, na verdade, era negativa.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- FN: False Negative - é a quantidade de erros quando tivemos uma previsão negativa, mas que, na verdade, era positiva.

Com métricas assim: falso, verdadeiro, positivo, negativo, fica um tanto confuso, não? Pois o nome da matriz é altamente sugestivo, não é?

4.2.2 Métricas de avaliação em classificadores

Da mesma forma que os regressores, para os classificadores temos muitas métricas para analisá-los. Veremos algumas delas e a resposta para a pergunta "qual métrica utilizar" não é tão trivial e, normalmente, é um sonoro "depende".

Vejamos algumas:

- Acurácia: em resumo, é a taxa de acertos, ou seja, de todas as previsões, qual percentual temos de acertos? A fórmula de cálculo é relativamente simples. ACC = (TP + TN) / (TP + TN + FP + FN).
- Precisão[11]: é uma das formas de avaliar os erros de previsão, neste caso, o erro FP. Sua fórmula de cálculo é P = TP / (TP + FP).
- Recall[12]: é uma das formas de avaliar os erros de previsão, neste caso o erro FN. Sua fórmula de cálculo é R = TP / (TP + FN).

No Código-fonte “Pipeline de um algoritmo preditivo em *Python*”, vimos o cálculo do accuracy_score, através da métrica (função) presente no sklearn. O cálculo das 3 métricas acima podemos visualizar na Figura “Métricas Accuracy, Precision e Recall”.

[11] Precision também é chamado de PPV ou Positive predictive value.
[12] Recall também é chamado de TPR ou True Positive Rate, Sensitivity ou, ainda, Hit rate.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
acc = accuracy_score(y_teste, y_prev)
p = precision_score(y_teste, y_prev)
r = recall_score(y_teste, y_prev)

[acc, p, r]
```

```
[0.678082191780822, 0.6086956521739131, 0.3853211009174312]
```

Figura 19 – Métricas Accuracy, Precision e Recall
Fonte: Elaborado pelo autor (2022)

Temos várias outras métricas de erro para classificadores, gráficos de transição (como a curva ROC). Porém, para o escopo desta disciplina, é suficiente que entendamos bem estas métricas.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

KELLEHER J. D.; TIERNEY B. ***Machine Learning***. New Jersey: MIT Press, 2018.

SMOLA A.; VISHWANATHAN S. V. N. ***Introduction to Machine Learning***. Cambridge: Cambridge University Press, 2008.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com